

MATEC: A lightweight neural network for online encrypted traffic classification[☆]

Jin Cheng^{a,b}, Yulei Wu^{c,*}, Yuepeng E^{b,*}, Junling You^b, Tong Li^b, Hui Li^b, Jingguo Ge^b

^a School of Cyber Security, University of Chinese Academy of Science, Beijing, China

^b Institute of Information Engineering, Chinese Academy of Science, Beijing, China

^c College of Engineering, Mathematics and Physical Sciences, University of Exeter, Exeter, EX4 4QF, UK

ARTICLE INFO

Keywords:

Traffic classification
Multi-head attention
Convolutional network
Lightweight model

ABSTRACT

Increased awareness of privacy protection has led to a surge in the volume of encrypted traffic, which creates a heavy burden for efficient network management (e.g. quality-of-service guarantees). The opacity of encrypted traffic essentially requires high computational overheads to make traffic classification, which is even worse when encrypted traffic surges. However, existing deep learning approaches sacrifice the efficiency to obtain high-precision classification results, which are no longer suitable for scenarios with large volumes of encrypted traffic. In this paper, a lightweight and online approach implemented as MATEC is proposed. The way we optimize the classification process follows the “Maximizing the reuse of thin modules” design principle. The multi-head attention and the convolutional network are adopted in the thin module. Attributed to the one-step interaction of all packets and the parallel computing of the multi-head attention mechanism, a key advantage of our model is that the number of parameters and running time are significantly reduced. In addition, the effectiveness and efficiency of convolutional networks have been proved in traffic classification. Comparisons to the existing state-of-the-art models on three typical datasets demonstrate that the proposed MATEC model has higher accuracy and running efficiency. In addition, the number of parameters is reduced to 1.8% of the state-of-the-art models and the training time halves.

1. Introduction

Traffic classification is a crucial tool for network management, such as congestion control and quality-of-service (QoS) guarantees [1]. Due to growing concerns about the privacy and security of Internet users, an increasing number of Internet applications have adopted various encryption technologies [2]. According to the statistics of Firefox, the global usage of encrypted web pages increased from 32% in 2015 to 83% in 2020.¹ Although encrypting transmitted data protects the users' privacy and security, it brings significant challenges for network management in terms of security and QoS [3,4]. For example, Voice over Internet Protocol (VoIP) applications encrypt the payload and leverage different methods to bypass firewalls and proxies [4], which challenges the intrusion detection systems. Besides, a pricing mechanism is needed to differentiate customers with different needs and charge according to the QoS that they have received [3]. It also

acts as a cost recovery mechanism and provides revenue generation for the Internet Service Providers (ISP) to compensate for their efforts in providing QoS and managing resource allocation. All QoS schemes have some traffic classification considerations in the design [3].

Real-time traffic classification is the core component of many emerging QoS-enabled products and automated QoS architectures [5, 6]. With the explosive growth of encrypted traffic, the implementation of QoS policies requires more efficient traffic classification models with lower overheads [7]. Consequently, heavyweight models for encrypted traffic classification cannot be applicable in real-time scenarios and high-speed networks. Nevertheless, existing traffic classification approaches sacrifice efficiency to guarantee the high precision of classification results. Most existing machine learning (ML) methods are based on the extraction of the hand-crafted statistical features [8–10], such as mean packet length, flow duration, and the mean inter-arrival

[☆] This work is supported by Special Project of Ministry of Science and Technology of China on Innovation Method (Grant No. 2019IM020100), Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No. XDC02070200), and the STS (Science and Technology Service Network) Plan of Chinese Academy of Sciences under Grant No. KFJ-ST-S-QYZD-2021-11-001.

* Corresponding author.

E-mail addresses: chengjin@iie.ac.cn (J. Cheng), y.l.wu@exeter.ac.uk (Y. Wu), eyuepeng@iie.ac.cn (Y. E), youjunling@iie.ac.cn (J. You), litong@iie.ac.cn (T. Li), lihui1@iie.ac.cn (H. Li), gejingguo@iie.ac.cn (J. Ge).

¹ <https://letsencrypt.org/stats/#percent-pageloads>.

time of the packet. However, observing the entire or large part of a flow is necessary to obtain these statistical features, so the ML method can only apply to the offline classification scenarios. Although traffic classification methods based on deep learning (DL) can eliminate the disadvantages of manually constructed features, some problems still exist. First, the features used are relatively homogenized [11,12]. In other words, most existing studies either use statistical features or packet byte features, which miss important information (i.e., these two kinds of features are not used in combination). Second, most existing models lack the consideration of model efficiency, where they neither take into account the time and space complexity nor fully evaluate the efficiency [2,13,14]. For DL methods, the huge overhead of memory and runtime in the complex neural network leads to high energy consumption [14–16].

In this paper, to raise the online performance and efficiency of encrypted traffic classification, the time and space complexity are taken into consideration. We propose a lightweight neural network called MATEC for online encrypted traffic classification, which takes three consecutive packets at a random location in a flow as the input. On the basis of the principle “maximizing the reuse of thin modules”, the features at global (flow) and local (packet) levels are extracted by using a thin module, which adopts multi-head attention [17] and 1D convolutional networks (1D-CNN) [13]. Generally, global features come from the interactions between packets in a flow, and local features are included in the bytes from a few raw packets. At the global level, with the multi-head attention [17], each packet interacts with all the other packets in one step to calculate the packet weights by multiplying matrices, greatly reducing the number of parameters and the running time. Additionally, through the multi-head attention mechanism, a packet is projected into multiple subspaces, and therefore different feature interactions in different subspaces can be captured in parallel. Therefore, the parameters of the multi-head attention mainly come from the projection matrix and is usually low. In contrast, in many other methods [2,11], recurrent neural networks (RNN) [18] are usually utilized to obtain global packet interactions by processing the packets in time series due to their ability in processing sequential data tasks. Whereas, only two neighboring packets can interact at a clip, making it time-consuming and inefficient due to the extra time overheads of waiting for the output from the previous interaction. At the local level, 1D-CNN [13] focuses on the packet bytes and is utilized to extract packet-level features, which has been proved with high effectiveness and efficiency in traffic classification [13,19,20]. More details about the time and space complexity are presented in Section 4.3.

Moreover, the traffic from zero-day applications usually does not exist in the training dataset [16]. For example, in a ground truth dataset [21,22], the zero-day applications can make up to 60% of flows and 30% of bytes, which refers to the presence of lots of new traffic. However, existing traditional methods [2,13,23] were trained on a specific dataset so that the new traffic cannot be classified. With these considerations in mind, we are inspired to use transfer learning [24] to solve the problem. For newly coming traffic, the previously trained model is fine-tuned based on a small amount of labeled data. The experimental results show that for newly coming traffic data, the previously trained model only needs to adjust the last layer of the neural network in structure to adapt to the new traffic and achieves rapid model convergence.

This work is based on our former conference paper [25]. However, the previous work ignored the traffic data outside of the dataset and did not take the similarities and differences among different datasets consideration. The model training of previous work was trained based on a specific dataset. Therefore, when new types of traffic data appear, the trained model cannot work and a new model need to be trained from scratch, which takes considerable time to converge. This journal version addresses this issue and provides the solution with transfer learning [24]. Besides, as more and more new traffic emerges, there is

no single dataset that can contain all traffic types. Three datasets [11, 26,27] are utilized to simulate the real-world traffic, and each dataset contains traffic types that the other datasets do not. Besides, more baselines, and evaluation metrics are utilized to carry out a more comprehensive validation of the proposed model in terms of efficiency and stability.

The main contributions of this paper can be summarized as follows:

- **A lightweight model called MATEC for online encrypted traffic classification is designed.** The proposed MATEC is an end-to-end model without feature engineering and subproblem partition, which guarantees the global optimality of the results. Only three consecutive packets at a random position are needed as input. Besides, the model automatically extracts high-order flow-level and packet-level features in an efficient way. Comparisons to the existing state-of-the-art models on three typical datasets demonstrate that the proposed MATEC model has higher accuracy and running efficiency. In addition, the number of parameters is reduced to 1.8% of the state-of-the-art models and the training time halves.
- **A novel approach to extract high-order flow-level features is proposed.** Existing methods apply RNN (e.g., LSTM and GRU) to extract the flow-level features, which is time-consuming and computationally complex. To overcome these shortage, we explore multi-head attention mechanism [17] to model the high-order packet interactions at the flow level. To the best of the authors' knowledge, it is the first time the multi-head attention is used in the field of encrypted traffic classification, which enables each packet to interact with all other packets in one step. Meanwhile, different interactions in different subspaces are learned in parallel, reducing the number of parameters and shortening the running time considerably. As the experimental results show, the number of parameters in our model is reduced to 2.7% of CNN with the Long Short-Term Memory (LSTM) model, and 19.8% of the Bi-directional Gated Recurrent Unit (Bi-GRU) model. Besides, the training time of our model only takes 6.8% and 35.8% of that of the above two RNN models.
- **A new embedding approach for packet representation is used,** which can facilitate the extraction of high-order features efficiently. Most existing studies either use statistical features or packet byte features, which miss important information for encrypted traffic classification. In our work, both the statistical features (e.g. the packet length and packet position) and the packet byte features are embedded into a vector uniformly to represent a packet (see Section 4.2.1). Experimental results demonstrate that our embedding method improves the precision of encrypted traffic classification by 1.7%.
- **Given the similarities and differences of datasets, we employ transfer learning to MATEC.** As more and more new types of traffic emerges, the traffic data outside of the dataset need to be considered. We utilized three datasets to simulate real traffic data and each dataset contains traffic types that the other datasets do not. By transfer learning [24], for a new dataset, our previously trained model only needs to adjust the last layer of the neural network to adapt to the new dataset and achieves rapid model convergence. Besides, even when the size of the new training data is relatively small (i.e., 10% of the overall dataset), the accuracy is still 8.04% higher than that of the model trained from scratch. Besides, the model has a better convergence speed.

The rest of this paper is structured as follows. Section 2 introduces the related work of the traffic classification. The preliminaries of work are presented in Section 3, and the proposed model MATEC is elaborated in Section 4. Section 5 launches the analysis of experimental results. Finally, Section 6 draws conclusion of this paper.

2. Related work

Our model involves three lines of works: (a) Encrypted traffic classification, (b) Flow-based methods, and (c) Packet-based methods, which will be introduced next in this section. Meanwhile, the differences and improvements that are compared with the previous work will be summarized.

2.1. Encrypted traffic classification

Traffic classification aims to categorize the encrypted network traffic into appropriate classes [16]. The common process of traffic classification is shown in Fig. 1. First, the target of traffic classification should be defined clearly. The general goals include QoS provisioning, intrusion detection, malware detection, resource usage planning, and so on [16]. To meet its target, the traffic classification is usually carried out based on (1) protocols (e.g. HTTP, SSL, SMTP, DNS or QUIC), (2) traffic types (e.g. video, chat, or browsing), (3) applications (e.g. Amazon, Apple, Microsoft or Google), and (4) websites. For all the above four cases, labeling each flow with corresponding traffic classes is a necessity. In our work, we identify the application of the flow. Second, a large and representative dataset is required. Although a few public datasets are available for research, there is no agreed common dataset in the field of traffic classification [16]. In our work, two public datasets [26,27] and one dataset collected by ourselves from China Science and Technology Network (CSTNET) are utilized for model validation. Significantly, data in these datasets are raw and we labeled them with the advanced method [28]. Then, the features are selected or extracted by predefined feature processing methods. At last, the classification model will be trained, validated, and evaluated. Furthermore, traffic classification can also be categorized into two subclasses: online and offline. The method we proposed in this work is an online method, which only takes three consecutive packets at a random position in a flow as the input.

Besides, two types of granularity are usually considered in the field of traffic classification [16]: packet and flow, which will be covered in detail in Sections 2.2 and 2.3, respectively. In our case, we choose the flow granularity, which is common in both industry and academia [2]. A flow refers to all packets with the same values of the quintuple (i.e., transport layer protocol, source IP, source port, destination IP, destination port), where the source and the destination can be swapped [29]. For any flow f_i in the flow set F , we have

$$F = \{f_1, f_2, \dots\} \quad (1)$$

$$f_i = \{p_1^i, p_2^i, \dots, p_n^i\} \quad (2)$$

where flow f_i contains n packets, and packet p_j^i is the j th packet in f_i . We select N consecutive packets $P = \{p_k^i, \dots, p_{k+N-2}^i, p_{k+N-1}^i\}$ in the flow i at a random location. That is, k is an integer number in the range $[1, n - N + 1]$. In our work, we set $N = 3$, which indicates that three consecutive packets are chosen to represent the flow f_i . Formally, a classifier is trained to recognize the category to which a flow belongs (i.e., classifier: $f_i \rightarrow \text{class}$).

2.2. Packet-based methods

As shown in Fig. 2, the packet-based methods for traffic classification take the bytes of a packet as the input directly. The packet bytes include the information of the packet header and payload data. Table 1 shows a summary of the packet-based methods. With respect to the packet header, the information in layer 3 and layer 4 are useful for non-encrypted traffic [16]. Before the deep learning era, fields including port number, protocol, and packet length were carefully chosen by domain experts as representative features. In some recent approaches [2,30,42], entire packets are taken as the input. For payload

Table 1

A summary of existing traffic classification methods.

Methods	Ref.ID	Classifiers	Features
Packet-Based	[30]	Stacked autoencoder (SAE) and CNN	Packet payload
	[31]	Reinforcement Learning (RL)	Packet bytes
	[32]	Generative Adversarial Networks (GAN)	Deep embedding
	[33]	Autoencoder and K-Means	Packet bytes
	[34]	Autoencoder	Packet payload
Flow-Based	[35]	Kiss, a novel classification engine	Payload signatures
	[36]	Support vector machine (SVM)	Packet length and TCP flags
	[37]	Hidden Markov models (HMM)	Packet length and time delay
	[23]	Gaussian mixture models and HMM	Inner-packet time and packet size
	[11]	Gated Recurrent Unit (GRU)	Sequence of packets
	[13]	CNN and RNN	Sequence of packets
	[38]	Naive Bayes	Session duration and TCP port
	[39]	Expectation Maximum (EM)	Packet sizes and inter-arrival time
	[40]	Fuzzy C-Means	Sequence of packets
	[41]	K-means	Attribute vectors
	[2]	CNN and LSTM	Sequence of packet

data, the information above the layer 4 header can be exploited for traffic classification. In work [30], after an initial pre-processing phase, packets were fed into the model with stacked autoencoder and CNN to classify network traffic. Liang et al. [31] proposed a deep reinforcement learning approach to solve the packet classification problem. The method in [31] used succinct representations to encode state and action space, and efficiently explored candidate decision trees to optimize a global objective. Finamore et al. [35] proposed a classification approach that leveraged the statistical characterizations of the payload by extracting payload signatures. However, the packet-based methods focus on the detailed information of a few packets and lack attention to global characteristics. The packet-level features contain rich local information from the header and payload of the packet. Flow-level features show global information, such as statistical information. In our model, both packet-level and flow-level features are considered, and through a combination of the two kinds of features, our model can obtain richer feature information and achieve better performance, as demonstrated in Section 5.3.2.

2.3. Flow-based methods

As shown in Fig. 3, flow-based methods require the selection of flow features, and these features are then fed into the classifier. In general, there are two main classes: machine learning-based and deep learning-based. Table 1 also shows a summary of the flow-based methods.

2.3.1. Machine learning-based

Existing machine learning-based methods for traffic classification at the flow level are mainly based on complex manual features that rely on expert experience to design. For example, in work [43], Moore et al. designed about 250 flow features based on prior knowledge. Subsequent ML-based traffic classification works (e.g., [28,44,45]) are mostly based on these 250 features for feature extraction. These flow characteristics include, but are not limited to variance of packet bytes, median of the total bytes in IP packets, and so on. Zhu and Zheng [36] studied the application of support vector machine (SVM) to classify the network traffic by using the packet length and TCP flags, which converted the actual problem into the high-dimensional feature space through the kernel function. Fu et al. [37] built hidden Markov models (HMM) to classify the services of mobile messaging Apps by jointly modeling user behavioral patterns, network characteristics, and temporal dependencies. Along this line, the authors in [37] first segmented

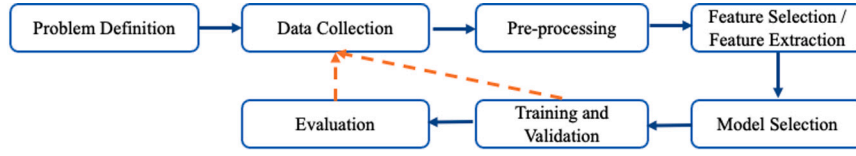


Fig. 1. A common process for traffic classification.



Fig. 2. A common framework of packet-based methods for traffic classification.

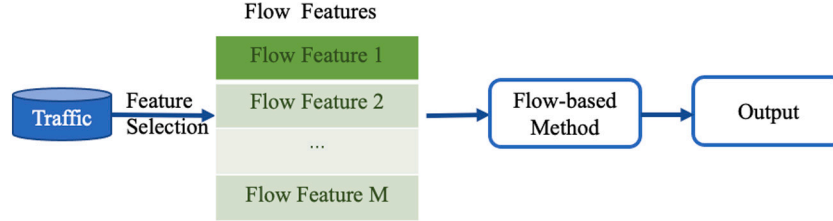


Fig. 3. A common framework of flow-based methods for traffic classification.

Internet traffic from traffic-flows into sessions with several dialogues in a hierarchical way, and two features including packet length and time delay are employed. According to the distribution of time and packet size, the authors in [23] classified the obfuscated traffic with Gaussian mixture models and HMM, which established the connection between inner-packet time and packet size to improve the accuracy of the flow feature description. However, these classifiers have to observe the whole flow or most of the packets in a flow to obtain these features, which are more appropriate for offline classification. In this paper, with only a few consecutive packets as the input, the proposed model can meet the real-time requirements in the field of online traffic classification.

2.3.2. Deep learning-based

Deep learning methods have been widely used in traffic classification. In [11], the bidirectional gated recurrent unit (Bi-GRU) was used to extract the forward and backward features of the byte sequences in a flow. Authors in [12] applied Bi-GRU to extract the hidden information in the packet length sequence. The study [13] took a few consecutive packets as the input of 1D-CNN, which was the first work to apply the end-to-end method on the traffic classification and achieved an accuracy of 85.8%. To get spatial and temporal features, both CNN and RNN were used in [46]. The study [46] first learned the spatial features of network traffic by using deep CNN and then learned temporal features by using the recurrent network. The authors in [2] proposed a model with 2D-CNN and LSTM networks, which accepts a few consecutive packets as the input. In [2], CNN was used to extract the packet features for a single packet and the LSTM network was trained to pick out the flow features based on the inputs of the packet features in a flow. As we can see above, pure CNN lacks the ability to extract interactive information, which leads to the fact that CNN often combines with RNN to extract the flow features. However, the dependencies between units in RNN make it time-consuming to wait for the output of the previous unit. In our proposed method, interactions of all packets are implemented directly by the application of multi-head attention [17].

2.4. The difference with our previous work

As mentioned, this work is based on our former conference paper [25]. However, the previous work ignored the traffic data outside

of the dataset and did not take the differences among datasets into consideration. The model of previous work was trained on a specific dataset and could not classify the newly coming traffic data. However, as more and more new types of traffic emerges, there is no single dataset that can contain all traffic types. In our work, we leverage three datasets [11,26,27] to simulate the real traffic data, and each dataset contains traffic types that the other datasets do not. If model training in different datasets is completely independent of each other, it will be time-consuming and unstable. This journal version addresses this important issue and puts forward a transfer learning solution. With the help of transfer learning, when new traffic arrives, the model can achieve fast convergence based on a small amount of labeled data.

To carry out a comprehensive evaluation of the proposed solution, different from the conference version, more datasets, more baseline models, and more evaluation metrics are utilized to verify the efficiency and effectiveness of MATEC. First, three datasets are employed to validate the accuracy of MATEC. Two of these datasets [26,27] are open encrypted datasets, and the third is a dataset [11] collected by ourselves from the backbone of the China Science and Technology Network (CSTNET) - an Internet service provider. Second, three advanced benchmark models [2,11,13] are compared with our model. Third, the proposed model is evaluated in terms of the well-known effectiveness metrics including accuracy, F1-score, precision, and recall rate. Besides, the training time and testing time of the model are also measured.

3. Problem definition

3.1. Packet interaction

More efficient global information can be extracted with high-order interaction between packets [47]. Given N packets as the input, the packet interaction can be defined as

$$g(x_1, x_2, \dots, x_N) \quad (3)$$

where x_i is the unified representation of packet p_i , and $g(\cdot)$ is a non-additive interactive function, such as outer product [48,49] and multiplication [50]. For instance, $x_1 \times x_2$ is an interactive feature concerning packet x_1 and packet x_2 . Traditionally, the meaningful high-order interactive features are extracted by RNN. However, it is very time-consuming to wait for the pairwise interactions between the joint

Table 2
Classification targets of different datasets.

Dataset	Applications, the classification targets
[26]	nexus.ensighten.com, r.nexac.com, s.amazonaws.com, pixel.quantserve.com, mc.yandex.ru, usu.openx.net, tpc.googlesyndication.com, bat.bing.com, t.co, www.google.fr, adserver.adtechus.com, ads.yahoo.com, pbs.twimg.com, d.adroll.com, connect.facebook.net, beacon.krxd.net, batr.msn.com, www.google-analytics.com, tags.tiqcdn.com, www.googletagmanager.com, coupang.com, statsg.doubleclick.net, bam.nr-data.net, ascii.jp, fonts.googleapis.com, sb.scorecardresearch.com, dnxstfruwz.cloudfront.net, fonts.gstatic.com, ib.adnxs.com, secure.adnxs.com, cdnjs.cloudflare.com, cdn.optimizely.com, www.facebook.com, optimizedby.rubiconproject.com, ssl.gstatic.com, s.adroll.com, www.livepartners.com, selfrepair.mozilla.org, googleadsg.doubleclick.net, ssl.google-analytics.com, www.vcommission.com, scontentxx.fbcdn.net, api.mixpanel.com, assets.adobedtm.com, www.google.com, spanalytics.yahoo.com, analytics.twitter.com
[11]	hpd.baidu.com, wx.sinaimg.cn, i.hdslb.com, sz.btfs.mail.ftn.qq.com, sdkxyajs-data.xycdn.com, tracker.wps.cn, ss.bdstatic.com, mp.weixin.qq.com, ss.baidu.com, long.open.weixin.qq.com, wenku.baidu.com, hm.baidu.com, s.qhupdate.com, browserkernel.baidu.com, clients.google.com, dw-online.ksosoft.com
[27]	nexus.ensighten.com, abs.twimg.com, pixel.quantserve.com, mc.yandex.ru, bat.bing.com, platform.twitter.com, ads.yahoo.com, d.adroll.com, connect.facebook.net, beacon.krxd.net, www.google-analytics.com, tags.tiqcdn.com, s.amazonaws.com, ib.adnxs.com, secure.adnxs.com, www.facebook.com, ssl.gstatic.com, assets.adobedtm.com, www.google.com

packets. Besides, the limited hidden unit structure in the RNN cannot efficiently show the interpretability of the high-order features. Hence, we propose to explore an approach that can efficiently discover the meaningful high-order interactive features, and meanwhile, all packet features can be projected into many sub-spaces, where each packet can interact with all packets in one step.

3.2. Transfer learning

In the real scenario, the traffic from zero-day applications usually does not exist in the training set [16]. The zero-day applications can make up to 60% of flows in a network traffic dataset [21], which refers to new traffic types. Therefore, as more and more new traffic emerges, there is no single dataset that can contain all traffic types. Existing traditional methods were trained on a specific dataset so that the new traffic cannot be classified.

In our work, three datasets [11,26,27] are utilized simulate the real-world traffic, and each dataset contains the traffic types that the other datasets do not. As shown in Table 2, the classification targets (i.e., types of applications) of different datasets are different and the model trained on a single dataset may not work well for other datasets. To clarify, we treat the classification of different datasets as different tasks. If the models are trained independently in different tasks, it will be time-consuming to train the model from scratch in all tasks. Given that there are certain similarities between different datasets [11,26,27], we are inspired to use transfer learning [24] to solve the problem. In our work, a new model for the new traffic data can be trained based on the previously trained model. First, only small changes in structure are made to the previously trained model as illustrated in Section 4.2.3. Then, based on the new traffic data, all the parameters are fine-tuned to make the model adapt to the new task. In this way, due to the prior information in the previous model, the number of the required labeled data and the training time are greatly reduced.

4. MATEC: The proposed lightweight model

The design of MATEC is presented in this section, as shown in Fig. 4. First, either online or offline traffic traces captured on the router are

fed as the input. After preprocessing, N consecutive packets in a flow are captured (Section 4.1). Then, the initial features of each packet are extracted and embedded into several vectors that will be concatenated to represent the flow (Section 4.2.1). The attention encoder is employed to extract the local and global features (Section 4.2.2). At last, a fully connected layer with softmax is employed to output the probability distribution of labels (Section 4.2.3). Recall from Section 3.2 that transfer learning [24] is applied to optimize the training on new traffic data. In transfer learning, the fully connected layer is the only part that needs to be changed architecturally, because different targets of datasets cause the number of output layer units to be different. In other words, the structure of embedding and attention encoder layer is fixed in all tasks, and the structure of fully connected layer is variable. To enhance the stability of the lightweight model, a series of training optimization methods are adopted, including ResNet [51], layer normalization [52], and learning rate warmup [51]. In Section 4.3, we analyze the space and time complexity of our model.

4.1. Preprocessing

The hypertext transfer protocol secure (HTTPS) is a crucial encryption technology for network management, which is based on TCP [53]. In recent years, the number of applications using HTTPS has exploded [11]. In this work, HTTPS traffic is classified. The three-way handshake in TCP connections is skipped because it contains less information that is useful for traffic classification [54].

For the traffic data, we split the traffic into flows according to the quintuple of a flow. The N consecutive raw packets in a flow are extracted as the only inputs for our model.

4.2. The design of MATEC

4.2.1. Embedding

The embedding layer is used to extract the features of a packet at a shallow level. Unlike most other studies, our work utilizes both the statistical features and packet byte features. It is worth noting that the statistical features used are readily accessible, such as packet length and relative position, and can be obtained online.

As a result, the feature representations of packets are unified in the same format, which includes multiple feature fields in our model. For packet x_i , the shallow feature fields can be represented as:

$$x_i = \{x_{i_1}, x_{i_2}, \dots, x_{i_M}\} \quad (4)$$

where M is the number of feature fields, and x_{i_j} is the feature value of the j th field. For example, x_{i_1} represents the packet position (e.g., 1, 2, ..., N), x_{i_2} could denote the packet length, and x_{i_3} could be the first 784 bytes after removing the Ethernet header. It is worth noting that even for encrypted traffic, the information in the packet headers can be exploited for traffic classification [16]. Not only do we leverage the header of the packet, but a portion of the packet payload is also used. As shown in Fig. 5, the component of 784 bytes includes 20 bytes for the header of the IP layer (for IPv4 traffic only), 20 bytes for TCP/UDP header (where 12 zero bytes are used to complement the UDP header for alignment with the 20-byte TCP header), and the remaining 744 bytes for the packet payload (truncated if the payload exceeds 744 bytes, and conversely filled with zero bytes). For the number of packet bytes, a series of hyperparameter comparison experiments are conducted in Section 5. To speed up the data processing, the features are normalized to [0,1] by dividing by 255. Then, we unify these features. For a vectorial feature x_{i_j} , the dimension change can be expressed as

$$e_{i_j} = U_j x_{i_j} \quad (5)$$

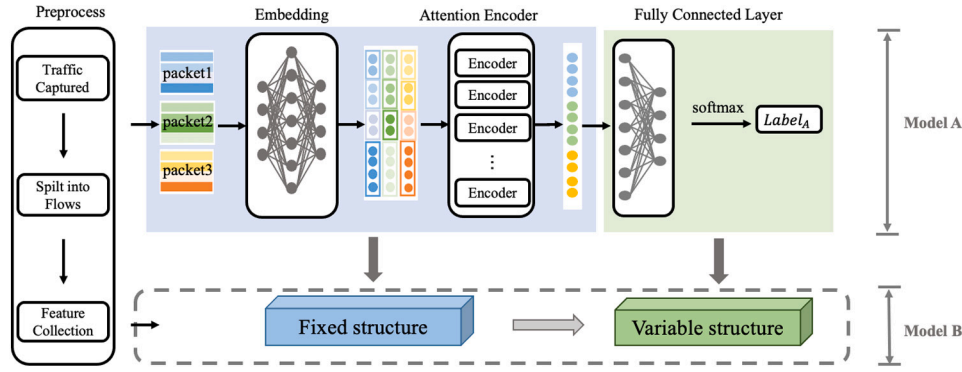


Fig. 4. The preprocessing and architecture of MATEC. The structure of the embedding and attention encoder layers is fixed, and the structure of fully connected layer varies with different tasks.



Fig. 5. The feature field of the packet bytes.

where i represents a packet and j denotes a feature field. U_j is an embedding matrix for field j , which can be predefined or trained. Otherwise, for a scalar feature x_{ij} , the dimension change can be expressed as

$$e_{ij} = u_j x_{ij} \quad (6)$$

where u_j is an embedding vector for field j . The above processing applies to all feature fields in each packet. It is worth noting that both parameters U_j and u_j are trainable.

Then, the vectorial and scalar features are transformed into a unified representation, and the packet vector e_i (take packet i as an example) can be represented as

$$e_i = W_{map}[e_{i1}, e_{i2}, \dots, e_{iM}] \quad (7)$$

where $[e_{i1}, e_{i2}, \dots, e_{iM}] \in \mathbb{R}^m$ is the result of concatenating all feature fields and $W_{map} \in \mathbb{R}^{d \times m}$ is the mapping matrix. Therefore, the packet can be expressed as a vector of d dimensions. The flow can be represented as a matrix with a shape of $N \times d$ by stacking the packet vectors, where N is the number of packets in a flow.

The position embedding is also carried out based on the method proposed in [17]. With the position encoded by Sines and Cosines as shown in (8), the method performs well in an NLP scene, where pos represents the location of the packet and i is the dimension. However, it does not work well for traffic classification, where absolute position encoding is more indicative of position information than relative position encoding. This can be solved by (6), i.e., multiplying the embedding vector u_j and position number x_{ij} .

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{model}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{model}}) \end{aligned} \quad (8)$$

4.2.2. Attention encoder

The attention encoder is designed to extract local and global features at a deeper level. Inspired by the literature [17], we take the advantage of thin modules with multi-head attention [17] and 1D-CNN feed-forward layers. Specifically, in our model, the thin module is reused T times to model the T -order interactions between the packets, and the number of parameters of the model is significantly reduced.

For the encoder, four lines of works are contained (see Fig. 6): (a) multi-head attention, (b) add & norm, (c) feed-forward layer and (d)

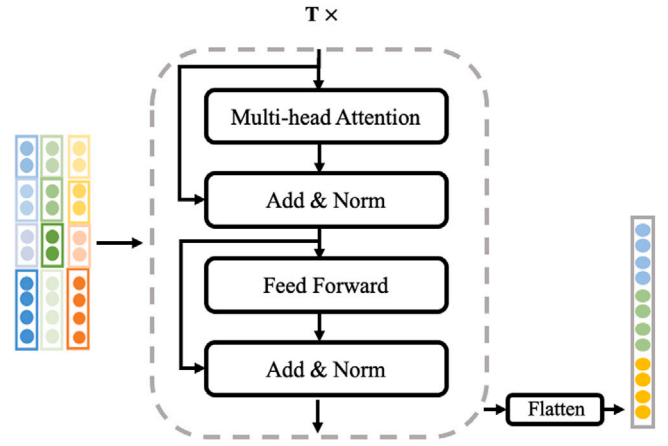


Fig. 6. The construct of attention encoder.

flatten layer. In what follows, the detailed design of each component will be explained.

(a) Multi-head Attention

The multi-head attention block is utilized to extract the flow-level features, as it can model the interactions between packets. In the field of traffic classification, multi-head attention has not been verified for its effectiveness. We find that it can be one of the most effective and efficient blocks to extract flow-level features, as shown by the experimental results in Section 5.3. With the help of multi-head attention, each packet is allowed to interact with all the other packets in one step as shown in (9), greatly reducing the number of parameters and shortening the running time. Moreover, a packet is projected into multiple subspaces as illustrated by (10) and (11), and therefore the multi-head attention can capture different feature interactions in different subspaces as shown in (12). By stacking the module with multi-head attention, we can model different orders of feature interactions.

Take packet m for an example, and we explain the extraction method for high-order features. First, the attention weight $\alpha_{m,k}^h$ between packet m and k is defined as

$$\alpha_{m,k}^h = \frac{\phi^h(e_m, e_k)}{\sum_{i=1}^N \phi^h(e_m, e_i)} \quad (9)$$

$$\phi^h(e_m, e_k) = \langle Q^h e_m, K^h e_k \rangle, Q^h, K^h \in \mathbb{R}^{d' \times d} \quad (10)$$

where h is the attention head, while Q^h and K^h are the transformation matrices, mapping original embedding space $\mathbb{R}^d$ into a new space $\mathbb{R}^{d'}$ in head h . $\phi^h(\cdot, \cdot)$ defines the relationship between the two packets, and it can be realized by inner product $\langle \cdot, \cdot \rangle$ or a neural network. Typically, we use inner product on account of the operation efficiency.

Table 3

Comparisons with other feature extraction methods, where n is the sequence length, d is the representation dimension, h is the number of heads of multi-head attention, and k is the kernel size of convolutions.

Method	Complexity per layer	Sequential operations	Maximum path length
Multi-head attention	$O(h \cdot n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent network	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional network	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

By mapping the initial embedding space $\mathbb{R}^d$ into another space $\mathbb{R}^{d'}$, the temporary representation of packet m is defined as

$$\tilde{e}_m^h = \sum_{i=1}^N \alpha_{m,i}^h (V^h e_i), V^h \in \mathbb{R}^{d' \times d} \quad (11)$$

where V^h is the transformation matrix in head h .

After splicing all the $\tilde{e}_m^h$ in all heads, the packet vector can be expressed as

$$\tilde{e}_m = \text{Concat}(\tilde{e}_m^1, \dots, \tilde{e}_m^H) \quad (12)$$

where H is the number of heads. It is remarkable that the computation of all the heads is parallelized, contributing to the efficiency of the model running speed.

Furthermore, we compare various aspects of multi-head attention to the recurrent and convolutional layers commonly used for feature extraction [17], which maps one sequence of symbol representations $x = [x_1, \dots, x_n]$ to another sequence of equal length $z = [z_1, \dots, z_n]$ with $x, z \in \mathbb{R}^d$. As noted in Table 3, the comparisons include total computational complexity per layer, the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required and the path length between long-range dependencies in the network. As shown in Table 3, a multi-head attention layer connects all positions with a constant number of sequentially executed operations, which can largely reduce the running time. However, a recurrent layer requires $O(n)$ sequential operations due to the cell unit dependence [11]. With respect to the computational complexity, multi-head attention layers are faster than recurrent layers when $h \times n < d$, which has been considered in our traffic classification tasks. Besides, convolutional networks also have good performance in terms of complexity per layer and sequential operations, which are used to extract packet-level features in Section 4.2.2(c).

(b) Add & Norm

$$e_m = \text{ReLU}(\tilde{e}_m + W_{res} e_m) \quad (13)$$

where $W_{res} \in \mathbb{R}^{d' \times d}$ (In our method, $d' H = d$), is a transformation matrix and $\text{ReLU} = \max(0, z)$ is a non-linear activation function.

However, changes in the output of one layer will cause highly correlated changes in the summed inputs to the next layer, especially with ReLU units which can significantly change the outputs [52], which may cause ‘‘covariate shift’’ problem [55]. Therefore, we apply layer normalization [52] to reduce the ‘‘covariate shift’’ problem [55] by fixing the mean and variance of the summed inputs within each layer. Therefore, the layer normalization over all the hidden units in the same layer (i.e., $Y^l = [y_1, y_2, \dots, y_H]$) is computed as follows,

$$\mu^l = \frac{1}{H} \sum_{i=1}^H y_i^l \quad \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (y_i^l - \mu^l)^2} \quad (14)$$

$$\tilde{Y}^l = \left[\frac{y_1 - \mu^l}{\sigma^l}, \frac{y_2 - \mu^l}{\sigma^l}, \dots, \frac{y_H - \mu^l}{\sigma^l} \right] \quad (15)$$

where H denotes the number of hidden units in a layer.

(c) Feed Forward Layer

In the Feed Forward layer, the deployment of 1D-Convolutional Neural Networks(1D-CNN) [13] helps to extract the packet-level features (see Fig. 7), as it focuses on a single packet. The details of the

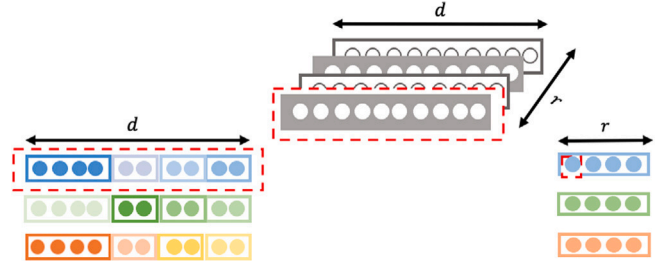


Fig. 7. The construct of feed-forward part of MATEC. The red dashed lines represent the units that are involved in generating a target unit.

convolutional networks have been shown in Table 3, which demonstrates the good performance of convolutional networks in terms of complexity per layer and sequential operations. In other words, the convolutional networks run with a constant number of sequentially executed operations and $O(\log_k(n))$ maximum path length, which proves the efficiency of 1D-CNN. In our work, a 1D-CNN layer with kernel size 1 is applied. For each packet, the inner products with each kernel constitute the deeper representation of the packet.

The combination of the three modules above (i.e., multi-head attention, add & norm, and feed-forward) makes up the loop block as shown in Fig. 6, with consistent dimensions of input and output.

(d) Flatten Layer

The flatten layer integrates the final vector of the flows. All the packet vectors (e_i) of the flow f are concatenated into a flatten vector $\tilde{E}_f$ to represent the flow f , shown in (13).

$$E_f = \text{Concat}(e_1, \dots, e_N) \quad (16)$$

4.2.3. Fully connected layer

Fully connected layer converts the flow feature E_f into the probability distribution of labels. The representation of the fully connected layer is shown as follows

$$\tilde{E}_f = \text{ReLU}(W_{Dense} E_f), W_{Dense} \in \mathbb{R}^{Nd \times R} \quad (17)$$

where R is the number of classification labels, and the output can be represented as $Z = [z_1, z_2, \dots, z_R]$. Then, the loss function of the model uses the cross-entropy function and can be defined as shown in (18),

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \text{loss} = \sum c_i \cdot \log(p_i) \quad (18)$$

where c_i is either 1 or 0, indicating whether the data label is c_i or not.

The structure of the fully connected layer is variable as shown in Fig. 4. As mentioned in Section 3.2, transfer learning is applied to optimize the training on new traffic data, which can speed up the convergence of the model. To clarify, we treat the classification of different datasets as different tasks. For different tasks, all fully connected layers have the same input dimension (i.e., Nd), but the output dimension is related to the number of classification targets, which varies with datasets. Due to the similarity between various datasets, the structure of embedding and attention encoder is fixed in all tasks, of which the parameters remain various common information. When trained on a new dataset, our model only needs to adjust the fully connected layer (i.e. the red part in Fig. 4) in structure, and fine-tune the previous parameters with few labeled data. In this way, the training time of the new tasks can be greatly reduced. It means that when new traffic data is available, the model can achieve rapid convergence with few labeled data. The effect of transfer learning is presented in Section 5.3.3.

4.3. Space and space complexity analysis

4.3.1. Space complexity

First of all, in the embedding layer, parameters are contained within the embedding transformation matrix and transformation vector. The

Table 4
Complexity of loop block of attention encoder.

Layer	Space complexity	Time complexity
Multi-head Attention	$O(Tdd')$	$O(TN^2dd')$
Add & Norm	$O(Tdr)$	$O(THdd')$
Feed Forward	$O(THdd')$	$O(TNd^2r)$
All	$O(THdd' + Tdr)$	$O(Tdd'(N^2 + H) + TNd^2r)$

Table 5
The details of the experiment configuration.

Item	Configuration
Hardware	Dell EMC PowerEdge R730, Intel(R) Xeon(R) CPU E5-2609
Operating system	Linux 5.0.0-32-generic # 18.04.2-Ubuntu
Memory	69G
Python version	Python 3.7.3
Tensorflow version	Tensorflow 1.14.0

number of the parameters is $M_1d_1d_2 + M_2d_3 + dm$, where M_1 represents the number of vectorial features fields, and M_2 denotes the number of scalar features fields. $d_1 \times d_2$ is the shape of the embedding transformation matrix and d_3 is the length of the embedding transformation vector of the scalar value. As mentioned in Section 4.2.1, d is the final dimension of the packet vector and m is the dimension of the intermediate result.

Second, for the attention encoder, the parameters mostly come from the loop module, which contains multi-head attention, add & norm module, and the feed-forward layer. The loop block contains the following weight matrices: $T \times \{Q^h, K^h, V^h, W_{res}, W_{CNN}\}$. Independent of the packet number N , the complexity is listed in Table 4. Besides, the parameter values are usually small as shown in Section 5.1.1.

4.3.2. Time complexity

For the embedding layer, it takes $O(M_1d_1d_2 + M_2d_3 + dm)$ to embed the original features into the unified representation. For the vectorial feature fields, the time complexity is $O(M_1d_1d_2)$, and for scalar feature fields, the time complexity is $O(M_2d_3)$. The representation of the parameter is described in the part of space complexity.

For the attention encoder, we take the loop block into consideration. The details of the time complexity of the loop block are shown in Table 4. In multi-head attention, it takes $O(Ndd' + N^2d')$ time for each head in the module to calculate the attention weight, and $O(N^2dd')$ time to form a combinatorial feature under one head. All the heads can be calculated in parallel. Therefore, the network is also time-efficient, because N, T, d' and r are usually small as shown in Section 5.1.1.

5. Experiments and performance analysis

5.1. Experiment setup

5.1.1. Experiment environment

The experiments are mainly performed on Dell EMC PowerEdge R730 with a Ubuntu operating system, and Table 5 shows the details of the experiment configuration. The hyperparameter settings of the MATEC are displayed in Table 6, and the experiments of hyperparameter comparisons are also conducted in this section. Besides, the training procedure is implemented by minimizing the sum objectives of categorical cross-entropy and the L2 regularization with a penalty of 0.05. The learning rate is 0.01, and the training batch size is 32. Moreover, we use Adam optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. To prevent the model from overfitting in the initial phase and ensure the model stability, warmup learning rate [51] is conducted. The number of warmup steps is set to be 4.

Table 6
The parameters of MATEC.

Notation	Parameter	Value
Number of packets	N	3
Number of loop blocks	T	2
Number of headers	H	3
Number of packet bytes	P	784
Dimension of embedding	d	432
/Kernel size of 1D-CNN		
Kernel number of 1D-CNN	r	432
Vector dimension of subspace	d'	144

5.1.2. Evaluation metrics

To evaluate the effectiveness of MATEC, we adopt four generally recognized metrics, namely accuracy, precision, recall, and F1-score. *Accuracy* represents the proportion of correctly classified flows. With respect to the other three metrics, we obtain the true positive (TP), false positive (FP), and false negative (FN) by comparing the output classification results with labeled ground truth. Take class i as an example, the information above can be used to calculate the index of *Precision_i*, *Recall_i*, and *F1-score_i* as the following formula show. The *Precision*, *Recall*, and *F1-score* are obtained by averaging the corresponding metrics of each class.

$$Precision_i = \frac{TP_i}{TP_i + FP_i} \quad (19)$$

$$Recall_i = \frac{TP_i}{TP_i + FN_i} \quad (20)$$

$$F1-score_i = \frac{2 \times Precision_i \times Recall_i}{Precision_i + Recall_i} \quad (21)$$

Additionally, to evaluate the efficiency of MATEC, we measure the training time and testing time per 100 batches, and the batch size is set to 32.

5.2. The overview of baselines and datasets

5.2.1. Baselines

To confirm the effectiveness of our model, we compare the model with the following baselines in terms of accuracy, precision, recall, and F1-score. For model efficiency, our model is compared with other models in terms of average training time and average testing time.

- 1D-CNN Model

This model was proposed in [13]. This method integrated feature selection, feature extraction, and classifier into a unified end-to-end framework with 1D-CNN, intending to automatically learn the nonlinear relationship between raw input and expected output. To the best of our knowledge, it was the first time to apply an end-to-end method in the encrypted traffic classification domain.

- CNN with LSTM Model

This model was introduced in [2] and employed to compare the effectiveness of multi-head attention [17] and RNN for flow feature extraction. It should be noted that in [2], the convolutional network was utilized to extract the packet features for a single packet. RNN (or LSTM) units were trained to extract the flow features, and the model also took three consecutive packets in a flow as the input.

- Bi-GRU model with transfer learning

The work was shown in [11], which also utilized RNN (Bi-GRU) to model the packet interactions. With respect to the input, the model selected three packets in a flow, where each packet contained 900 bytes. Besides, the attention mechanism was adopted to assign weights to features according to their contributions to the classification. The Bi-GRU network in [11] has been proved to be the state-of-art deep learning method for encrypted traffic classification.

Table 7
The details of Dataset B.

Applications	Flows	Applications	Flows
hpd.baidu.com	2598	wx.sinaimg.cn	2114
i.hdslb.com	1806	sz.btfs.mail.ftn.qq.com	2558
sdkeyajsdta.xycdn.com	6754	tracker.wps.cn	1992
ss.bdstatic.com	2494	mp.weixin.qq.com	1724
ss.baidu.com	2908	long.open.weixin.qq.com	11 056
wenku.baidu.com	2160	hm.baidu.com	18 694
s.qhupdate.com	2874	browserkernel.baidu.com	1666
clients.google.com	1148	dwnonline.ksofsoft.com	2128

5.2.2. Datasets

Three datasets are employed to validate the performance of MATEC. Two of them (Dataset A and Dataset C) are public datasets, which contain flows in all directions (i.e., from client to server and from servers to clients). One dataset (Dataset B) only contains the flow from clients to servers, which is collected by ourselves from China Science and Technology Network (CSTNET). In our work, dataset A and B are utilized to verify the effectiveness of the MATEC, and Dataset C is used to verify the effectiveness of transfer learning. Detailed information about these datasets has been shown in Table 2 in Section 3.2. The datasets are briefly described as follows:

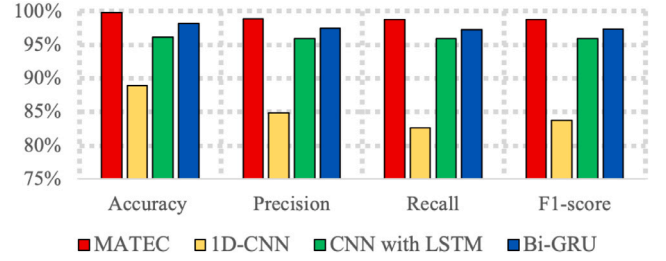
- Dataset A [27] was published by the University of Lorraine, constructed by crawling HTTPS websites over two weeks. The dataset contains PCAP files of crawling top 779 accessed HTTPS websites by Google Chrome and Mozilla Firefox Web browsers. To simplify the training process, we removed the special characters (e.g. dashes and numbers). At last, the applications with more than 1000 flows were chosen to use in the experiment, which contains 47 classes and 84 245 flows.
- Dataset B [11] was collected by ourselves from the backbone of CSTNET, an internet service provider in China. This dataset only contains unidirectional traffic flows, from clients to servers. In the end, the top 16 services with the most traffic flows were selected as the experimental data, which contained more than 49K flows. The details of Dataset B are shown in Table 7.
- Dataset C [26] is exploited to verify the efficiency and effectiveness of transfer learning. The dataset was published by Stratosphere Labs, supported by the CTU University of Prague in the Czech Republic. The Lab continuously publishes new network traffic datasets of malicious, mixed, and normal traffic. In our work, “CTU-Normal-20” to “CTU-normal-32” normal captures are used, including HTTPS raw traffic generated by Frantisek Strasak. Finally, the top 19 applications with the most traffic data were selected to verify the efficiency and effectiveness of the model.

It is important to note that all the original data is raw, and it is necessary to label the data manually. To label the data, we adopt an advanced labeling method proposed in [28], and its effectiveness has been verified in several works [11,12,28]. The proposed labeling process can be summarized as (1) getting the value of server name indication (SNI), which is the domain's substring of applications, (2) comparing IP address resolution with the SNI field for the credibility enhancement, and (3) discarding the flow if the SNI field is null or the IP cannot be parsed into a domain name. In theory, the accuracy of the label is doubly verified, including parsing the IP address to the corresponding domain name and confirming the domain name with the SNI value, which can guarantee the validity of the label.

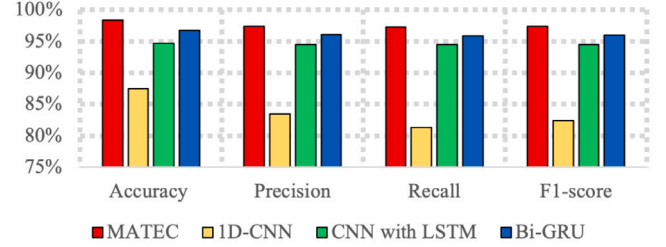
5.3. Evaluation on three datasets

5.3.1. Evaluation on dataset A

- Effectiveness analysis



(a) MATEC with the embedding method we illustrate in Section 4.2.1



(b) MATEC that only embeds the packet bytes as the original feature like the other three baseline works.

Fig. 8. Dataset A: comparisons between MHCNET and the three baseline models on accuracy, precision, recall and F1-score.

To validate the merits of MATEC, we compare MATEC with three well-known models as we described above (a) 1D-CNN [36], (b) CNN (2D) with LSTM model [13], and (c) Bi-GRU [11]. We retrieve the three benchmark models with the same architectures in the works [13,36] and [11], and the batch size is uniformly set to 32.

As mentioned above, a new embedding approach for packet representation is explored, which can enhance the efficiency of extracting high-order features. Both the statistical features (e.g. the packet length and packet position) and the packet byte features are embedded into a vector uniformly to represent the packet. While in the other three works, packet bytes are the only original features that are input into the model. As presented in Fig. 8, a comparative experiment of the two embedding methods is conducted on Dataset A. The experimental results are generated from 10-fold cross-validation. As shown in Fig. 8(a), when the embedding method we illustrate in Section 4.2.1 is adopted, it is obvious that our model outperforms the baseline models on all metrics. The accuracy, precision, recall, and F1-score rate reached 99.76%, 98.84%, 98.74%, and 98.79%, respectively. Improvements of 15.01%, 2.89%, and 1.38% on F1-score compared to the 1D-CNN, CNN with LSTM and Bi-GRU model demonstrate the great performance of our model. As shown in Fig. 8(b), when only the packet bytes are embedded as the original feature like the other three baseline works, the performance of all models degrades due to less information available. The experimental results show that our embedding method improves the precision of MATEC by 1.7%. To summarize, the model performs better when the embedding method we propose is applied.

The model which employs 1D-CNN to classify flows performs worst due to the oversimplified model structure, which focuses on extracting packet-level features. However, this work inspires us to explore an end-to-end approach for traffic classification. The second benchmark model utilizes both the packet features and flow features as considered in our model, where CNN extracts the packet-level features and LSTM is used to obtain the flow-level features. Experimental results show that LSTM is not as effective as multi-head attention in the extraction of flow features. This is because the multi-head attention is attributed to the one-step interaction of all packets and the parallel computing of all subspaces. Owing to the use of multi-head attention, a key advantage of our model is that the number of parameters and running time are significantly reduced. Therefore, the multi-head attention mechanism

Table 8

The comparison results of efficiency.

Item	MATEC	1D-CNN	CNN with LSTM	Bi-GRU
Weights	102k	5779k	3754k	514k
Average training time (ms)	24.9	50.1	434.5	69.5
Average testing time (ms)	10.0	13.2	114.8	29.0

Table 9

Details of different devices.

Number	Operating systems	CPU	Memory
1	Ubuntu 14.04 LTS	2 × Intel(R) Xeon(R) CPU E5-2620 v3	69G
2	Ubuntu 14.04 LTS	2 × Nvidia Tesla K80	69G
3	Mac 11.3.1	6 × Intel Core i7-9750H @ 2.60 GHz	32G
4	Windows 10 Pro	6 × Intel Core i7-8700 @ 4300 MHz	16G

is proved to be effective to extract features in high dimensions. Besides, our model also can benefit from the application of ResNet, layer normalization, and learning rate warmup. The Bi-GRU model performs better than the other two baselines, which utilizes an attention mechanism to assign greater attention to more important features. Same as the LSTM, the Bi-GRU model can only handle the interaction of two packets at a clip. As a result, it is time-consuming and inefficient to wait for the output from the previous interaction in the RNN model due to the extra time overheads required.

- Efficiency analysis

To achieve the best lightweight performance, we adopt some principles, which can be summarized as (a) reuse of thin modules, (b) paralleled computation in multiple heads (i.e., subspaces), and (c) multiple training optimization techniques, like ResNet, layer normalization, and learning rate warmup.

First, the efficiency of the model is evaluated in terms of the model weights, training time and testing time. As shown in Table 8, MATEC benefits greatly from the principles mentioned above. The number of parameters of MATEC is 1.8% of 1D-CNN, 2.7% of CNN with LSTM, and 19.8% of the Bi-GRU model. With respect to time efficiency, training and testing time of each 100 batches are considered as metrics. The training time of MATEC is 49.7% of 1D-CNN, 6.8% of CNN with LSTM, and 35.8% of the Bi-GRU model. It should be noted that LSTM and Bi-GRU can also be conducted to extract flow-level features, but only two adjacent packets can interact at a time in LSTM, which leads to high time consumption.

Second, we verify the online performance of the classifier. Online traffic classification systems mainly consist of two parts, traffic capture part and traffic classification part [56]. However, the authors in [57] indicated that traffic classifier is the bottleneck of the network traffic classification system because traffic capture has already reached a very high speed due to the development of FPGA [56]. Therefore, we can conclude that the online performance of the classifier is determined by the throughput of the classifier, and we measure the throughput of the models on different devices. The details of devices are shown in Table 9, and the testing results are depicted in Fig. 9. In this figure, the horizontal axis represents the different devices, and the vertical axis denotes the number of flows that can be processed per second. As we can see, our model has the highest processing efficiency on different devices. On device 3, the throughput of MATEC (our model), 1D-CNN, CNN with LSTM, and Bi-GRU models were 9568 flows/s, 5376 flows/s, 768 flows/s, and 2912 flows/s, respectively. The throughput of our model is about twice than that of the 1D-CNN model and 12 times than that of the CNN with LSTM model.

- The comparisons on hyperparameters.

Furthermore, we conduct experiments on Dataset A to explore the effects of different hyperparameters on the performance of MATEC.

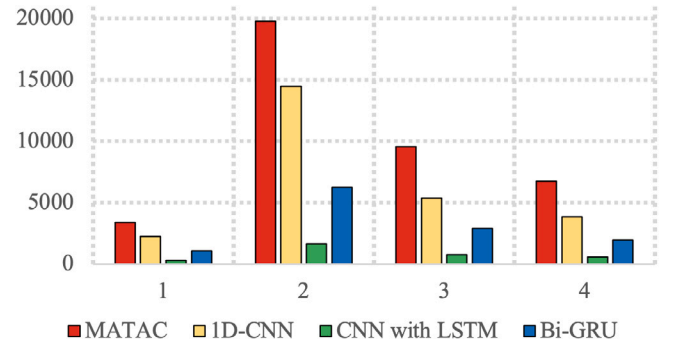


Fig. 9. Online performance measured on different devices. The meaning of the number values is shown in Table 9.

Table 10

Dataset B: comparisons on F1-score with the three baseline models. The values 1–16 are the same as the Fig. 12.

ID	MATEC	1D-CNN	CNN with LSTM	Bi-GRU
1	94.63%	78.90%	84.99%	93.47%
2	90.07%	79.00%	83.33%	88.59%
3	87.91%	58.85%	71.86%	88.07%
4	99.98%	97.89%	98.87%	99.98%
5	99.41%	95.50%	97.18%	99.40%
6	99.98%	93.12%	98.73%	99.98%
7	90.47%	63.23%	82.76%	81.63%
8	91.74%	71.10%	88.89%	87.36%
9	90.59%	75.73%	85.86%	85.85%
10	99.28%	95.60%	97.72%	98.47%
11	94.91%	90.38%	94.74%	94.44%
12	97.45%	93.24%	95.60%	97.14%
13	96.89%	79.08%	93.78%	95.39%
14	99.70%	91.03%	96.53%	98.82%
15	100.00%	99.60%	99.98%	100.00%
16	98.53%	96.79%	96.69%	99.01%

The main parameters include N , T , H , and P . N denotes the number of packets that are selected in a flow, T represents the number of the loop blocks (i.e., the attention encoder) in the model, H is the number of the headers in the multi-head attention, and P represents the number of packet bytes. More detailed meanings of the parameters have been illustrated in Section 4. We use the parameter values in Table 6 as default hyperparameters, where $N = 3$, $T = 2$, $H = 3$ and $P = 784$. For each experiment, we change only one parameter and keep the default values for the other parameters. As shown in Fig. 10(a), the performance of our model is sensitive to the value of N . When N is three, the model achieves an accuracy of 99.76%, and there is a dramatic increase in accuracy when the number of packets increases from one to two. As shown in Fig. 10(b), the accuracy of our model reaches 99%, when the parameter T is 2. After increasing the value of T , the accuracy increases by 1%. This indicates that the model with two loop blocks is good enough for extracting packet-level and flow-level features. In Fig. 10(c), when the number of headers in multi-head attention is three, the model shows the optimal performance. As the number of heads increases, the model becomes less effective. This is because the increase in the number of headers leads to a further refinement of feature granularity, which can undermine the integrity of the flow features. Fig. 10(d) indicates that the first 784 bytes of the packet contain information that is effective enough for encrypted traffic classification, and the increase of bytes has little effect on the performance of the model. The first 784 bytes of a packet contain (a) 20 bytes for the header of IP layer (for IPv4 traffic only), (b) 20 bytes for TCP/UDP header (where 12 zero bytes are used to complement the UDP header for alignment with the 20-byte TCP header), and (c) the remaining 744 bytes for the packet payload (truncated if the payload exceeds 744 bytes, and conversely filled with zero bytes).

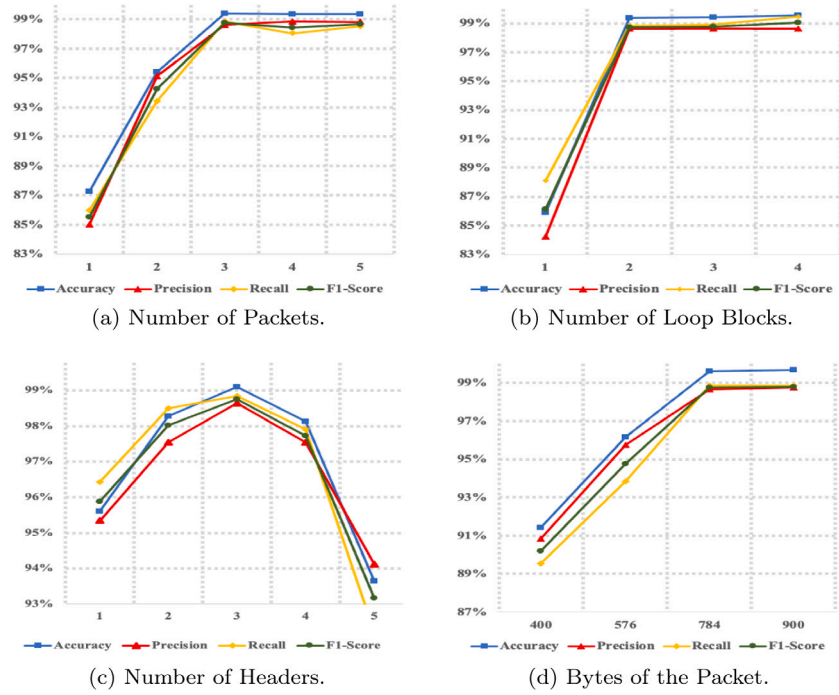


Fig. 10. Effects of different hyperparameter values on experimental results.

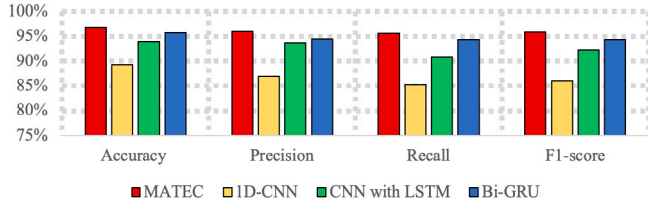


Fig. 11. Dataset B: comparisons between MATEC and the three baseline models on accuracy, precision, recall and F1-score.

5.3.2. Evaluation on dataset B

To enhance the validation of MATEC, we also conduct an experiment on Dataset B as we do on Dataset A, in terms of accuracy, precision, recall, and F1-score. As shown in Fig. 11, the accuracy, precision, recall and F1-score of MATEC reach 96.84%, 95.98%, 95.53% and 95.75%, respectively. Improvements of 10.22%, 3.85%, and 1.49% on F1-score compared to the 1D-CNN, CNN with LSTM, and the Bi-GRU models give more evidence that our model performs well for encrypted traffic classification.

Fig. 12 contains three subgraphs, which display the precision, recall, and F1-score values of 16 classes in Dataset B. In Fig. 12(a), the precision values for 10 classes are greater than 95%, and the average precision of MATEC is greater than that of the other three models by 9.12%, 2.33%, and 1.56% respectively. As shown in Fig. 12(b), for the class “s.qhupdate.com”, MATEC achieves a recall of 96.89%, with improvements of 17.81%, 3.10%, and 1.50%, respectively, compared to the other three. In Fig. 12(c) and Table 10, the F1-scores of 5 classes are greater than 99%. However, all models perform poorly on the “i.hdslb.com”, because there are only 1806 flows in this class. Through the experiment in this section, it can be concluded that for services from different providers, MATEC can achieve excellent performance.

5.3.3. Effectiveness of transfer learning

Dataset C is exploited to verify the effectiveness of transfer learning along with Dataset A. First, a source model is trained on Dataset A. Then, only small changes (i.e. the fully connected layer) in structure

are made to the trained model as illustrated in Section 4.2.3 and we initialize the parameters of fully connected layer randomly. At last, based on dataset C, all the parameters are fine-tuned to make the model adapt to the new task. In this way, due to the prior information in the previous model, the number of the required labeled data and the training time are greatly reduced.

We conduct a set of experiments with the training data accounting for 10%, 30%, 60%, 80%, 100% of the total data in dataset C. The experimental results of Dataset C are presented in Figs. 13 and 14. These figures demonstrate the effects of the number of training samples on the accuracy, precision, recall, F1-score, and training speed of MATEC. Overall, the model based on the transfer learning performs better in terms of the evaluation metrics mentioned above. When the proportion of training data is relatively small (i.e., 10%), the accuracy, precision, recall, and F1-score is 8.04%, 13.97%, 7.81%, and 11.00% higher than those of the model trained from scratch. This is because when the number of training samples is small, the information in the training data is insufficient to represent the real data distribution. The relevant prior knowledge from the source model can compensate for this shortcoming. Furthermore, for a new task, the convergence speed of the model is significantly increased with the help of transfer learning. As shown in Fig. 13, in the context of a small number of training samples, the weak generalization ability of the model trained from scratch results in lower accuracy, precision, recall, and F1-score. In contrast, models that have been trained on a previous dataset can use prior knowledge to assist in improving accuracy significantly. When the proportion is relatively large, the gap between the two methods shrinks due to the increasing learning ability of the model trained from scratch. Moreover, Fig. 14 shows that, as the training set grows larger, training a completely new model takes much more time.

5.4. Remarks

In this section, we compare MATEC with the other three baseline models in terms of effectiveness and efficiency. For Dataset A, three groups of comparison experiments are conducted, including effectiveness comparison, efficiency comparison, and hyperparameter comparison. According to the effectiveness analysis, improvements of

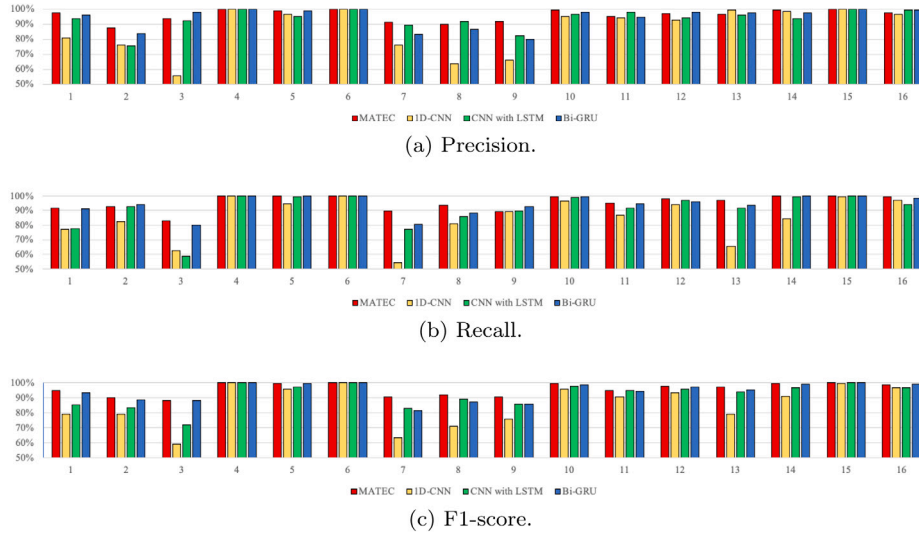


Fig. 12. Dataset B: comparisons with the three baseline models. The values of 1 to 16 on the x axis represent the classes of 'hpd.baidu.com', 'wx3.sinaimg.cn', 'il.hdslb.com', 'sz.bfss.mail.ftn.qq.com', 'sdk1xyajs-data.xyedn.com', 'tracker3.wps.cn', 'ss0.bdstatic.com', 'mp.weixin.qq.com', 'ss0.baidu.com', 'long.open.weixin.qq.com', 'wenku.baidu.com', 'hm.baidu.com', 's.qhupdate.com', 'browserkernel.baidu.com', 'clients1.google.com', 'dw-online.ksofsoft.com'.

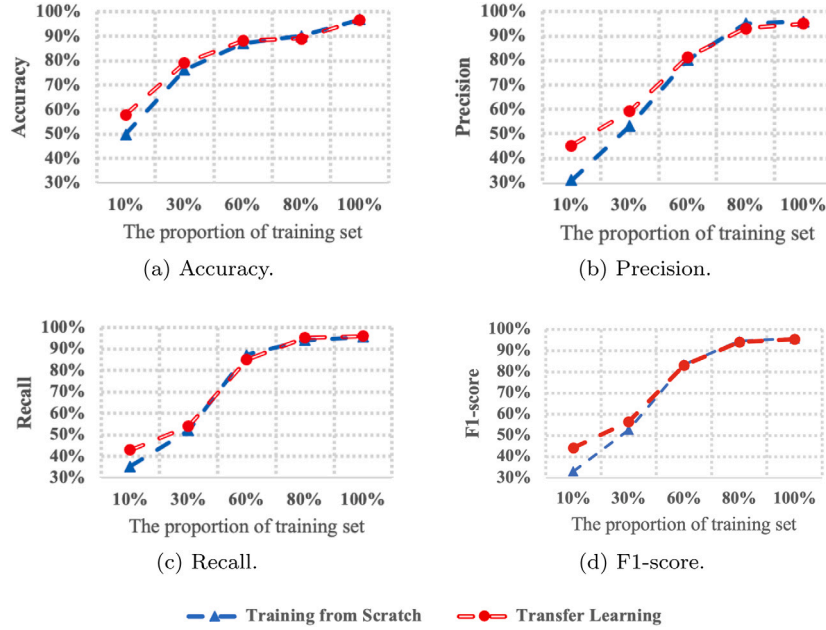


Fig. 13. Evaluation of transfer learning. Parts (a), (b), (c), and (d) represent the changes in accuracy, precision, recall, F1-score, and training time, respectively, with increasing amounts of training data.

15.37%, 2.87%, and 1.77% on the F1-score rate compared to the other three models demonstrate that our model performs well for encrypted traffic classification. For the efficiency analysis, the performance of our model achieves the optimal in terms of the number of parameters, the training speed, and testing speed. More specifically, the number of parameters of our model is 1.8% of 1D-CNN, 2.7% of CNN with LSTM, and 19.8% of the Bi-GRU model, and the training time of our model is 49.7% of 1D-CNN, 6.8% of CNN with LSTM, and 35.8% of Bi-GRU model. At last, the comparisons on hyperparameters check the optimum configuration of MATEC. Dataset B is utilized to enhance the validation of the model's effectiveness. As shown in Table 10 and Fig. 12, the precision values for 10 classes are greater than 95%, and the average precision of MATEC is greater than that of the other three models by 9.11%, 2.33%, and 1.56%, respectively. Besides, the advantages of transfer learning are tested on Dataset C. For the model trained with the transfer learning, even when the proportion of training data is relatively

small, the accuracy, precision, recall, and F1-score are higher than those of the model trained from scratch. Furthermore, the convergence speed of the model is significantly improved by the transfer learning. In short, MATEC is excellent in terms of effectiveness and efficiency.

6. Conclusion

Burning requirements on the efficiency of traffic classification are put forward due to the widespread adoption of encrypted traffic. In this paper, a lightweight model called MATEC has been proposed to reduce resource and time consumption. "Maximizing the reuse of thin modules" is our design principle. A thin module has been adopted the multi-head attention and the 1D convolutional network. Attributed to the one-step interaction of all packets and the parallel computation of the multi-head attention mechanism, a key advantage of MATEC

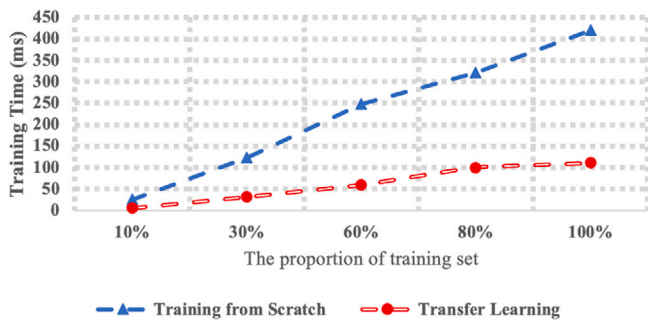


Fig. 14. Evaluation of transfer learning on training time.

is that the number of parameters and the training and testing time are significantly reduced. To improve the stability of the model, the designed network is trained with the aid of ResNet, layer normalization, and learning rate warmup. The proposed model outperformed the state-of-the-art works based on deep learning on three datasets. The results has shown that our model has higher accuracy and running efficiency, while the number of parameters used is reduced to 1.8% of that in the 1D convolutional network and the training time halves on the single task. By transfer learning, for a new dataset, our trained model only needs to adjust the last layer of the neural network to adapt to the new dataset and achieves rapid model convergence. Besides, even when the size of the training data is relatively small (i.e., 10% of the overall dataset), the accuracy is still 8.04% higher than that of the model trained from scratch. Besides, the model has a better convergence speed.

In our work, a flow refers to all packets with the same values of the quintuple [transport layer; source IP; source port; destination IP; destination port]. However, during the mobile session, the port number may change. In terms of packet information for classification, new protocols such as HTTP2 and HTTP3 use a new flow identifier (i.e., Globally Unique Identifier) [58] to identify user sessions. In the future work, we will consider using the Global Unique Identifier to identify flow instead of the quintuple.

CRediT authorship contribution statement

Jin Cheng: Conceptualization, Methodology, Software. **Yulei Wu:** Writing - original draft. **Yuepeng E:** Visualization, Investigation. **Junling You:** Validation. **Tong Li:** Software. **Hui Li:** Data curation, Writing - review & editing. **Jingguo Ge:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

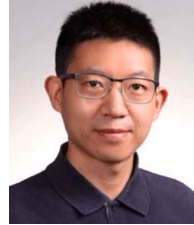
- [1] H.-N. Dai, R.C.-W. Wong, H. Wang, Z. Zheng, A.V. Vasilakos, Big data analytics for large-scale wireless networks: Challenges and opportunities, *ACM Comput. Surv.* 52 (5) (2019) 1–36.
- [2] Z. Zou, J. Ge, H. Zheng, Y. Wu, C. Han, Z. Yao, Encrypted traffic classification with a convolutional long short-term memory neural network, in: 2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), IEEE, 2018, pp. 329–334.
- [3] T.T. Nguyen, G. Armitage, A survey of techniques for internet traffic classification using machine learning, *IEEE Commun. Surv. Tutor.* 10 (4) (2008) 56–76.
- [4] A.S. Khatouni, N. Zincir-Heywood, Integrating machine learning with off-the-shelf traffic flow features for http/https traffic classification, in: 2019 IEEE Symposium on Computers and Communications (ISCC), IEEE, 2019, pp. 1–7.

- [5] L. Stewart, G. Armitage, P. Branch, S. Zander, An architecture for automated network control of QoS over consumer broadband links, in: TENCN 2005-2005 IEEE Region 10 Conference, IEEE, 2005, pp. 1–6.
- [6] J. But, N. Williams, S. Zander, L. Stewart, G. Armitage, Angel-automated network games enhancement layer, 2006.
- [7] M.F. Audah, T.S. Chin, Y. Zulfadzli, C.K. Lee, K. Rizaluddin, Towards efficient and scalable machine learning-based QoS traffic classification in software-defined network, in: International Conference on Mobile Web and Intelligent Information Systems, Springer, 2019, pp. 217–229.
- [8] W. Niu, Z. Zhuo, X. Zhang, X. Du, G. Yang, M. Guizani, A heuristic statistical testing based approach for encrypted network traffic identification, *IEEE Trans. Veh. Technol.* 68 (4) (2019) 3843–3853.
- [9] J. Meng, L. Yang, Y. Zhou, Z. Pan, Encrypted traffic identification based on sparse logistical regression and extreme learning machine, in: Proceedings of ELM-2014 Volume 2, Springer, 2015, pp. 61–70.
- [10] R. Alshammari, A.N. Zincir-Heywood, Identification of VoIP encrypted traffic using a machine learning approach, *J. King Saud Univ. Comput. Inform. Sci.* 27 (1) (2015) 77–92.
- [11] X. Liu, J. You, Y. Wu, T. Li, L. Li, Z. Zhang, J. Ge, Attention-based bidirectional gru networks for efficient https traffic classification, *Inform. Sci.* 541 (2020) 297–315.
- [12] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
- [13] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), IEEE, 2017, pp. 43–48.
- [14] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapé, Mobile encrypted traffic classification using deep learning, in: 2018 Network Traffic Measurement and Analysis Conference (TMA), IEEE, 2018, pp. 1–8.
- [15] F. Pacheco, E. Exposito, M. Gineste, A framework to classify heterogeneous internet traffic with machine learning and deep learning techniques for satellite communications, *Comput. Netw.* (2020) 107213.
- [16] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.
- [18] L.R. Medsker, L. Jain, Recurrent neural networks, *Des. Appl.* 5 (2001).
- [19] Z. Chen, K. He, J. Li, Y. Geng, Seq2img: A sequence-to-image based approach towards IP traffic classification using convolutional neural networks, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 1271–1276.
- [20] S. Rezaei, X. Liu, How to achieve high classification accuracy with just a few labels: A semi-supervised approach using sampled packets, 2018, arXiv preprint arXiv:1812.09761.
- [21] J. Zhang, X. Chen, Y. Xiang, W. Zhou, J. Wu, Robust network traffic classification, *IEEE/ACM Trans. Netw.* 23 (4) (2014) 1257–1270.
- [22] H. Kim, K.C. Claffy, M. Fomenkov, D. Barman, M. Faloutsos, K. Lee, Internet traffic classification demystified: myths, caveats, and the best practices, in: Proceedings of the 2008 ACM CoNEXT Conference, 2008, pp. 1–12.
- [23] Z. Yao, J. Ge, Y. Wu, X. Lin, R. He, Y. Ma, Encrypted traffic classification based on Gaussian mixture models and Hidden Markov models, *J. Netw. Comput. Appl.* (2020) 102711.
- [24] L. Torrey, J. Shavlik, Transfer learning, in: Handbook of Research on Machine Learning Applications and Trends: Algorithms, Methods, and Techniques, IGI global, 2010, pp. 242–264.
- [25] J. Cheng, R. He, Y. E. Y. Wu, J. You, T. Li, Real-time encrypted traffic classification via lightweight neural networks, in: IEEE Global Communications Conference, IEEE, 2020, pp. 1–6.
- [26] Stratosphere, Stratosphere laboratory datasets, 2015, Retrieved March 13, 2020, from <https://www.stratosphereips.org/datasets-overview>.
- [27] J.F. Wazen Shbair, T. Chole, Https websites dataset, 2016, 4http://betternet.lhs.loria.fr/datasets/https/.
- [28] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, L. He, Mampf: Encrypted traffic classification based on multi-attribute Markov probability fingerprints, in: 2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS), 2018, pp. 1–10, <http://dx.doi.org/10.1109/IWQoS.2018.8624124>.
- [29] A. Dainotti, A. Pescapé, K.C. Claffy, Issues and future directions in traffic classification, *IEEE Netw.* 26 (1) (2012) 35–40.
- [30] M. Lotfollahi, M.J. Siavoshani, R.S.H. Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Soft Comput.* 24 (3) (2020) 1999–2012.
- [31] E. Liang, H. Zhu, X. Jin, I. Stoica, Neural packet classification, in: Proceedings of the ACM Special Interest Group on Data Communication, ACM, 2019, pp. 256–269.
- [32] Y. Zhang, S. Zhao, Y. Sang, Towards unknown traffic identification using deep auto-encoder and constrained clustering, in: International Conference on Computational Science, Springer, 2019, pp. 309–322.

- [33] S. Zhao, Y. Zhang, Y. Sang, Towards unknown traffic identification via embeddings and deep autoencoders, in: 2019 26th International Conference on Telecommunications (ICT), IEEE, 2019, pp. 85–89.
- [34] Z. Wang, The applications of deep learning on traffic identification, *BlackHat USA 24* (11) (2015) 1–10.
- [35] A. Finamore, M. Mellia, M. Meo, D. Rossi, Kiss: Stochastic packet inspection classifier for udp traffic, *IEEE/ACM Trans. Netw.* 18 (5) (2010) 1505–1515.
- [36] Y. Zhu, Y. Zheng, Traffic identification and traffic analysis based on support vector machine, *Neural Comput. Appl.* (2019) 1–9.
- [37] J. Fu, H. Xiong, X. Lu, J. Yang, C. Chen, Service usage classification with encrypted internet traffic in mobile messaging apps, *IEEE Trans. Mob. Comput.* 15 (11) (2016) 2851–2864.
- [38] A.W. Moore, D. Zuev, Internet traffic classification using bayesian analysis techniques, in: Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2005, pp. 50–60.
- [39] A. McGregor, M. Hall, P. Lorier, J. Brunskill, Flow clustering using machine learning techniques, in: *International Workshop on Passive and Active Network Measurement*, Springer, 2004, pp. 205–214.
- [40] D. Liu, C.-H. Lung, P2P traffic identification and optimization using fuzzy c-means clustering, in: 2011 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE 2011), IEEE, 2011, pp. 2245–2252.
- [41] J. Erman, A. Mahanti, M. Arlitt, I. Cohen, C. Williamson, Semi-supervised network traffic classification, in: Proceedings of the 2007 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2007, pp. 369–370.
- [42] R. Li, X. Xiao, S. Ni, H. Zheng, S. Xia, Byte segment neural network for network traffic classification, in: 2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS), IEEE, 2018, pp. 1–10.
- [43] A. Moore, D. Zuev, M. Crogan, Discriminators for use in flow-based classification, 2013.
- [44] Y. Okada, S. Ata, N. Nakamura, Y. Nakahira, I. Oka, Comparisons of machine learning algorithms for application identification of encrypted traffic, in: 2011 10th International Conference on Machine Learning and Applications and Workshops, 2, IEEE, 2011, pp. 358–361.
- [45] G.-L. Sun, Y. Xue, Y. Dong, D. Wang, C. Li, An novel hybrid method for effectively classifying encrypted traffic, in: 2010 IEEE Global Telecommunications Conference GLOBECOM 2010, IEEE, 2010, pp. 1–5.
- [46] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, M. Zhu, Has-ID: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection, *IEEE Access* 6 (2017) 1792–1806.
- [47] X. He, T.-S. Chua, Neural factorization machines for sparse predictive analytics, in: Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2017, pp. 355–364.
- [48] J. Lian, X. Zhou, F. Zhang, Z. Chen, X. Xie, G. Sun, Xdeepfm: Combining explicit and implicit feature interactions for recommender systems, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, in: KDD '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1754–1763, <http://dx.doi.org/10.1145/3219819.3220023>.
- [49] R. Wang, B. Fu, G. Fu, M. Wang, Deep & cross network for ad click predictions, in: Proceedings of the ADKDD'17, ACM, 2017, pp. 1–7.
- [50] S. Rendle, Factorization machines, in: *IEEE International Conference on Data Mining*, IEEE, 2011, pp. 0–6.
- [51] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
- [52] J.L. Ba, J.R. Kiros, G.E. Hinton, Layer normalization, 2016, arXiv preprint [arXiv:1607.06450](https://arxiv.org/abs/1607.06450).
- [53] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, P. Steenkiste, The cost of the “s” in https, in: Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies, 2014, pp. 133–140.
- [54] A. Este, F. Gringoli, L. Salgarelli, Support vector machines for TCP traffic classification, *Comput. Netw.* 53 (14) (2009) 2476–2490.
- [55] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International Conference on Machine Learning*, PMLR, 2015, pp. 448–456.
- [56] A. Monemi, R. Zarei, M.N. Marsono, Online netfpga decision tree statistical traffic classifier, *Comput. Commun.* (2013).
- [57] H. Loo, S. Joseph, M.N. Marsono, Online incremental learning for high bandwidth network traffic classification, *Appl. Comput. Intell. Soft Comput.* 2016 (2016) 1–13, <http://dx.doi.org/10.1155/2016/1465810>.
- [58] R.S.P. Leach, M. Mealling, A universally unique identifier (UUID) URN namespace, 2005, RFC 4122 <https://www.hjp.at/doc/rfc/rfc4122.html>.



Jin Cheng received a B.Eng degree in software engineering from Xiamen University, Xiamen, China, in 2019. She is currently an M.Sc student at Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include future network architecture, network management, and network security and privacy.



Yulei Wu is a Senior Lecturer with the Department of Computer Science, College of Engineering, Mathematics and Physical Sciences, University of Exeter, United Kingdom. He received the B.Sc. degree (First Class Honors) in Computer Science and the Ph.D. degree in Computing and Mathematics from the University of Bradford, United Kingdom, in 2006 and 2010, respectively. His expertise is on intelligent networking, and his main research interests include computer networks, networked systems, software defined networks and systems, network management, and network security and privacy. He serves as an Associate Editor for *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Network Science and Engineering*, and *IEEE Access*, as well as an Area Editor for *Computer Networks* (Elsevier). He is a Senior Member of the IEEE and the ACM, and a Fellow of the HEA (Higher Education Academy).



Yuepeng E is an Associate Professor at Institute of Information Engineering, Chinese Academy of Sciences (CAS). He received his Ph.D. degree in Computer Software and Theory at the Graduate University of Chinese Academy of Sciences in 2012. His research interests include future network architecture and network testbeds.



Junling You is a Senior Engineer at Institute of Information Engineering, Chinese Academy of Sciences. Her interests are mainly in future network, network virtualization, and big data processing.



Tong Li is a Senior Engineer at Institute of Information Engineering, Chinese Academy of Sciences. He got a Ph.D. degree from the University of Chinese Academy of Sciences in 2015. His interests are mainly in network virtualization, network measurement and big data stream processing.



Hui Li is a Senior Engineer at Institute of Information Engineering, Chinese Academy of Sciences. He got an M.Eng degree from University of Chinese Academy of Sciences in 2006. His work focuses on computer network architecture, traffic analysis and cyber security.



Jingguo Ge received the Ph.D. degree in computer system architecture from the Institute of Computing Technology, Chinese Academy of Sciences, in 2003. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, and a Professor with the School of Cyber Security, University of Chinese Academy of Sciences. His research focuses on computer network architecture, software defined network (SDN), network virtualization, cyber security, and mobile communication networks.